

Pandas: мини-шпаргалка для экзамена по анализу данных

Содержание

0. Базовый шаблон	1
1. Быстрый осмотр данных	1
2. Выбор строк и столбцов	2
3. Сортировка, переименование, удаление	2
4. Типы данных и преобразования	2
5. Пропуски	3
6. Создание признаков	3
7. Агрегации и groupby	3
8. Сводные таблицы, concat, merge	4
9. Корреляции, выбросы, простые графики	4
10. Категории для ML	4
11. Мини-шаблон sklearn с pandas	5
12. Частые экзаменационные паттерны	5
13. Типовые ошибки	6
14. Мини-чеклист перед ответом	6

0. Базовый шаблон

```
import numpy as np
import pandas as pd
```

```
pd.set_option('display.max_columns', 100)
pd.set_option('display.width', 160)
```

```
df = pd.read_csv('data.csv')          # sep=';', encoding='utf-8'
# df = pd.read_excel('data.xlsx')
```

Часто на экзамене нужно не писать идеальный production-код, а быстро получить точное число. Перед ответом проверяй типы, пропуски, размер таблицы и уникальные значения.

1. Быстрый осмотр данных

```
df.shape          # (строки, столбцы)
df.head(3); df.tail(3)
```

```
df.info() # типы + non-null
df.describe() # числовые признаки
df.describe(include='object') # категориальные признаки
df.columns
df.dtypes
df.index
```

Пропуски и дубликаты:

```
df.isna().sum() # число NaN по столбцам
df.isna().mean().sort_values() # доля NaN
df.duplicated().sum()
df.drop_duplicates()
```

Уникальные значения:

```
df['col'].unique()
df['col'].nunique()
df['col'].value_counts() # частоты
(df['col'].value_counts(normalize=True) # доли
 .round(3))
```

2. Выбор строк и столбцов

```
df['age'] # один столбец: Series
df[['age', 'income']] # несколько столбцов: DataFrame

df.loc[0, 'age'] # по меткам индекса/столбцов
df.iloc[0, 2] # по позициям
df.loc[:, ['age', 'income']]
df.iloc[:10, :3]
```

Фильтрация:

```
df[df['age'] >= 18]
df[(df['age'] >= 18) & (df['income'] > 50000)]
df[(df['city'] == 'Moscow') | (df['city'] == 'SPb')]
df[df['city'].isin(['Moscow', 'SPb'])]
df[df['name'].str.contains('ann', case=False, na=False)]
df[df['col'].notna()]
```

Важно: в pandas используй &, |, ~, а не and, or, not. Условия обязательно бери в скобки.

3. Сортировка, переименование, удаление

```
df.sort_values('income')
df.sort_values('income', ascending=False)
df.sort_values(['city', 'income'], ascending=[True, False])

df.rename(columns={'old': 'new'}, inplace=False)
df.drop(columns=['bad_col'])
df.drop(index=[0, 1])
df.reset_index(drop=True)
```

4. Типы данных и преобразования

```
df['age'] = df['age'].astype(int)
df['income'] = pd.to_numeric(df['income'], errors='coerce')
df['date'] = pd.to_datetime(df['date'], errors='coerce')
df['cat'] = df['cat'].astype('category')
```

Работа с датами:

```
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['weekday'] = df['date'].dt.day_name()
df['days_from_start'] = (df['date'] - df['date'].min()).dt.days
```

Строки:

```
df['city'] = df['city'].str.lower().str.strip()
df['text_len'] = df['text'].str.len()
df['has_word'] = df['text'].str.contains('word', case=False, na=False)
```

5. Пропуски

```
df['age'].fillna(df['age'].median())
df['city'].fillna('unknown')
df.fillna({'age': df['age'].median(), 'city': 'unknown'})
df.dropna() # удалить строки с любым NaN
df.dropna(subset=['target']) # удалить строки, где NaN в target
```

Групповое заполнение, если медиана должна считаться внутри категории:

```
df['income'] = df.groupby('city')['income'].transform(
    lambda s: s.fillna(s.median())
)
```

6. Создание признаков

```
df['income_per_age'] = df['income'] / df['age']
df['is_adult'] = (df['age'] >= 18).astype(int)
df['income_level'] = np.where(df['income'] > 50000, 'high', 'low')
```

Несколько условий:

```
conditions = [
    df['age'] < 18,
    df['age'].between(18, 64),
    df['age'] >= 65
]
choices = ['child', 'adult', 'senior']
df['age_group'] = np.select(conditions, choices, default='unknown')
```

Функция по строке:

```
df['new'] = df.apply(lambda row: row['a'] + row['b'], axis=1)
```

Но если можно, используйте векторные операции без apply: они быстрее и обычно проще.

7. Агрегации и groupby

```
df['income'].mean()
df['income'].median()
df['income'].std()
df['income'].min(), df['income'].max()
df['income'].quantile(0.25)
```

Группировки:

```
df.groupby('city')['income'].mean()
df.groupby('city')['income'].agg(['count', 'mean', 'median', 'max'])
df.groupby(['city', 'gender'])['income'].mean()
```

Именованные агрегации:

```
res = df.groupby('city').agg(  
    n_clients=('client_id', 'count'),  
    avg_income=('income', 'mean'),  
    med_age=('age', 'median')  
) .reset_index()
```

Фильтрация после агрегации:

```
res[res['n_clients'] >= 100]
```

Добавить групповую статистику в исходную таблицу:

```
df['city_avg_income'] = df.groupby('city')['income'].transform('mean')
```

8. Сводные таблицы, concat, merge

Сводная таблица:

```
pd.pivot_table(  
    df,  
    values='income',  
    index='city',  
    columns='gender',  
    aggfunc='mean'  
)
```

Склеить строки или столбцы:

```
pd.concat([df1, df2], axis=0, ignore_index=True) # добавить строки  
pd.concat([df1, df2], axis=1)                  # добавить столбцы
```

Соединение таблиц:

```
df.merge(info, on='client_id', how='left')  
df.merge(info, left_on='id', right_on='client_id', how='inner')
```

Типы how: left - сохранить левую таблицу; inner - только совпадения; outer - все строки; right - сохранить правую.

9. Корреляции, выбросы, простые графики

```
df[['age', 'income', 'target']].corr(numeric_only=True)  
df['income'].corr(df['target'])
```

Частый способ найти выбросы через IQR:

```
q1 = df['income'].quantile(0.25)  
q3 = df['income'].quantile(0.75)  
iqr = q3 - q1  
low = q1 - 1.5 * iqr  
high = q3 + 1.5 * iqr  
outliers = df[(df['income'] < low) | (df['income'] > high)]
```

Быстрые графики:

```
df['income'].hist()  
df.boxplot(column='income', by='city')  
df.plot.scatter(x='age', y='income')
```

10. Категории для ML

Быстрое one-hot-кодирование:

```
df_ml = pd.get_dummies(df, columns=['city', 'gender'], drop_first=False)
```

Для sklearn лучше разделять признаки и целевую переменную:

```
X = df.drop(columns=['target'])
y = df['target']
```

```
cat_cols = X.select_dtypes(include=['object', 'category']).columns
num_cols = X.select_dtypes(include='number').columns
```

Минимальный вариант без Pipeline:

```
X = pd.get_dummies(X, columns=cat_cols, drop_first=False)
X = X.fillna(X.median(numeric_only=True))
```

11. Мини-шаблон sklearn с pandas

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, mean_absolute_error, r2_score
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
```

```
X = df.drop(columns=['target'])
y = df['target']
X = pd.get_dummies(X, drop_first=False)
X = X.fillna(X.median(numeric_only=True))
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = RandomForestClassifier(random_state=42) # классификация
# model = RandomForestRegressor(random_state=42) # регрессия
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

```
accuracy_score(y_test, pred) # классификация
# mean_absolute_error(y_test, pred) # регрессия
# r2_score(y_test, pred)
```

12. Частые экзаменационные паттерны

Количество строк по условию:

```
(df['age'] >= 18).sum()
```

Доля строк по условию:

```
(df['age'] >= 18).mean()
```

Среднее значение среди отфильтрованных строк:

```
df.loc[df['city'] == 'Moscow', 'income'].mean()
```

Город с максимальным средним доходом:

```
df.groupby('city')['income'].mean().idxmax()
```

Максимальное среднее значение:

```
df.groupby('city')['income'].mean().max()
```

Топ-5 категорий:

```
df['city'].value_counts().head(5)
```

Корреляция целевой переменной со всеми числовыми признаками:

```
df.corr(numeric_only=True)['target'].sort_values(ascending=False)
```

13. Типовые ошибки

1. `df[df['a'] > 1 & df['b'] < 5]` - неверно. Нужно `df[(df['a'] > 1) & (df['b'] < 5)]`.
2. `and/or` с Series не работают. Нужны `&` и `|`.
3. После `groupby` иногда нужен `.reset_index()`, иначе группировочный столбец станет индексом.
4. `inplace=True` не обязателен. Часто безопаснее писать `df = df.drop(...)`.
5. `mean()` чувствительно к выбросам, `median()` устойчивее.
6. `fillna(df.mean())` нельзя применять к строковым столбцам без отбора числовых.
7. Перед ML категориальные признаки нужно закодировать, а пропуски заполнить.
8. Тестовую выборку нельзя использовать для подбора правил обработки, если задача проверяет корректную ML-процедуру.

14. Мини-чеклист перед ответом

- Понял, что является строкой, признаком и целевой переменной.
- Проверил `shape`, `info`, `isna().sum()`.
- Проверил типы столбцов: числа не лежат строками.
- Для категорий посмотрел `value_counts()`.
- Для чисел посмотрел `describe()`.
- При фильтрации использовал скобки вокруг условий.
- При агрегации понял, нужен ли `count`, `mean`, `median`, `sum`, `max` или `idxmax`.
- В ML отдельно выделил X и y, закодировал категории, заполнил пропуски, сделал train/test split.